AI Receptionist

Group 5 CSCN8010 -

- Anthony Izevbokun, 9016626
- Preeja Anilal, 8791796
- Minh Thuan Truong, 8730956

Executive Summary

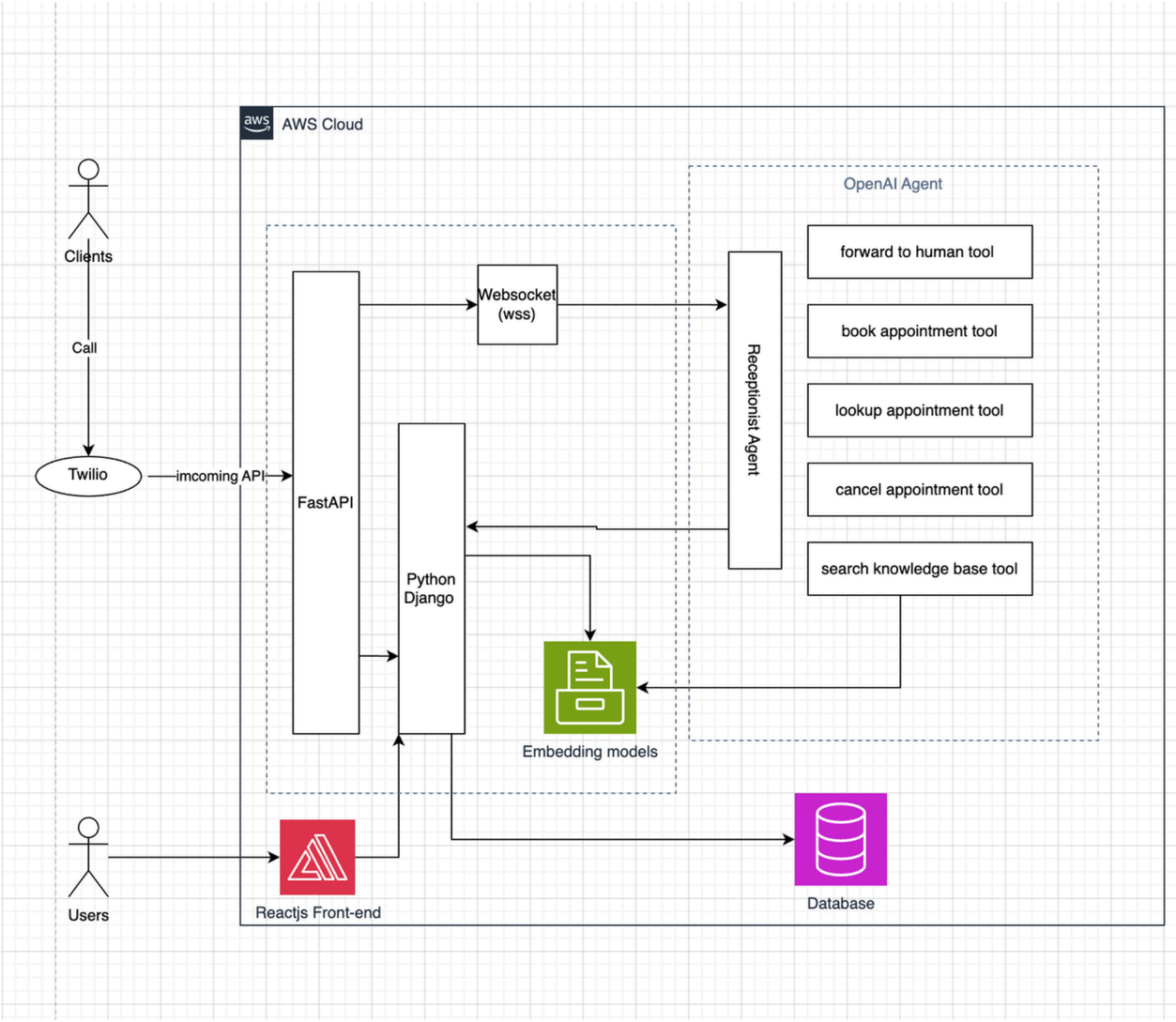
The Smart AI Receptionist is a real-time voice AI system

- Answers incoming phone calls, understands caller intent
- Detects emotional distress, and either resolves queries autonomously or escalates to a human agent.
- A DVC-orchestrated ML pipeline that trains post-call classifiers (category, sentiment, outcome) using TF-IDF + Logistic Regression and produces reproducible artifacts.
- A Build Knowledge workflow that collects business data, generates embeddings, stores vectors with metadata, and retrieves top-k business facts at runtime for grounded responses.

The system doesn't just talk. It listens, understands, classifies, and decides.

System Architecture

Speech-to-Speech (Realtime) Architecture



Technology Stack

Deliverable 1. NLP Model for Query Processing

- The NLP layer is the first point of intelligence in the pipeline.
- The moment a caller's speech is transcribed (via OpenAI Whisper / Realtime API), the text is passed through a natural language processing pipeline to extract structured meaning before any decision-making occurs.

- Intent Recognition The NLP model identifies what the caller wants — their primary intent. Examples:
- general_question — "What are your hours?"
- Named Entity Recognition (NER) Key entities are extracted from the utterance to parameterize the response:
  - Person names — "My name is John Smith"
  - Dates & times — "Next Tuesday at 3pm"

Implementation Approach

```python
# Simplified NLP pipeline within the agent tool call
def process_query(transcript: str, context: dict) -> dict:
    intent = classify_intent(transcript)          # Intent classification
    entities = extract_entities(transcript)        # SpaCy / OpenAI function calling
    slots = fill_slots(intent, entities, context)  # Dialogue state manager
    return {"intent": intent, "entities": entities, "slots": slots}
```

The NLP layer is embedded directly into the **OpenAI Agents SDK tool definitions**, where function-calling acts as the structured extraction mechanism — the LLM both understands and formats output simultaneously.

An handler is used to collect the NER entities and saved to our database. see code below

```python
    async def _realtime_session_loop(self) -> None:
        """Receive events from OpenAI RealtimeSession and forward audio to Twilio."""
        try:
            async for event in self._session:
                event_type = event.type

                if event_type == "audio":
                    await self._handle_audio_event(event)

                elif event_type == "audio_interrupted":
                    await self._handle_interruption()

                elif event_type == "audio_end":
                    pass

                elif event_type == "history_updated":
                    await self._handle_history_updated(event)

                elif event_type == "history_added":
                    pass  # handled via history_updated which has transcripts

                elif event_type == "agent_end":
                    logger.info("Agent ended session")

                elif event_type == "error":
                    logger.error(f"Realtime session error: {event.error}")

        except asyncio.CancelledError:
            pass
        except Exception as e:
            logger.error(f"Error in realtime session loop: {e}")
        finally:
            self._done_event.set()
```

Technology Stack

Deliverable 2 — Multi-Agent Handoffs & Tool Execution

- The running system is not a single monolithic model.
- A triage “AI Receptionist” agent (OpenAI Agents SDK) routes the caller, then hands off to specialized sub-agents.
- Each sub-agent can invoke Django-backed tools for real CRUD and lookups so answers stay consistent with the database.

```
    return RealtimeAgent[CallContext](
        name="AI Receptionist",
        instructions=instructions,
        tools=TRANSFER_TOOLS,
        handoffs=[
            realtime_handoff(
                faq_agent,
                tool_description_override="Transfer to the FAQ Agent for business hours, location, or service prices or
appointment lookup.",
            ),
            realtime_handoff(
                reschedule_agent,
                tool_description_override="Transfer to the Reschedule Agent for appointment rescheduling.",
            ),
            realtime_handoff(      You, 7 hours ago • - init project …
                booking_agent,
                tool_description_override="Transfer to the Booking Agent for appointment booking and service details.",
            ),
        ],
    )
```

Technology Stack

Deliverable 3 — Post-Call Classification (ML)

Our production flow now supports analysis backend for post-call classification after the call ends:

- ML backend (ml) using locally trained sklearn models
- It returns the structured fields and populate:
- outcome (successful | unsuccessful | unknown)
- sentiment (positive | negative | neutral)
- category (make_appointment | cancel_appointment | reschedule_appointment | ask_question | unknown)
- summary

```python
def analyze_conversation_ml(conversation: List[Dict[str, Any]]) -> Dict[str, str]:      You, 4 hours ago • - add DVC and
    """
    Predict outcome, sentiment, and category; summary is extractive (no LLM).
    Return shape matches OpenAIService.analyze_conversation.
    """
    text = conversation_to_text(conversation)
    if not text.strip():
        return {
            "outcome": "unknown",
            "sentiment": "neutral",
            "category": "unknown",
            "summary": "No transcript available.",
        }

    models = _get_models()
    row = [text]
    category = str(models["category"].predict(row)[0])
    sentiment = str(models["sentiment"].predict(row)[0])
    outcome = str(models["outcome"].predict(row)[0])
    return {
        "outcome": outcome,
        "sentiment": sentiment,
        "category": category,
        "summary": _fallback_summary(conversation),
    }
```

Technology Stack

Deliverable 4 — Build Knowledge Base (Embeddings + Vector Search)

We build a business-specific knowledge base so the receptionist can answer factual questions from current business data (services, hours, policies, contact, and FAQs) instead of relying only on prompt memory.

1. Collect business data from Django models and API endpoints.
2. Normalize and chunk text into retrieval-friendly documents.
3. Generate embeddings for each chunk.
4. Store vectors + metadata in a vector index.
5. Retrieve top-k context at runtime and inject into the agent/tool response.

Data Sources

- Business profile and contact details
- Opening hours and special hours
- Active services, prices, durations
- Appointment and cancellation policies
- Frequently asked questions and curated answers

```python
def reindex(self, reason: str, source_types: list[str] | None = None) -> dict[str, Any]:      You, 7
    collector = BusinessKnowledgeCollectorService(self.business)
    all_chunks = collector.collect()
    all_chunks = all_chunks[: self.MAX_CHUNKS_PER_REINDEX]
    if source_types:
        invalid = set(source_types) - self.ALLOWED_SOURCE_TYPES
        if invalid:
            raise ValueError(f"Unsupported source_types: {sorted(invalid)}")
        all_chunks = [chunk for chunk in all_chunks if chunk["source_type"] in source_types]

    candidate_keys = {(chunk["source_type"], chunk["source_id"]) for chunk in all_chunks}
    print(f"Candidate keys: {candidate_keys}")
    print(f"Source types: {source_types}")
    # print(f"Reason: {reason}")
    print(f"Business: {self.business}")
    existing = self.chunk_model.objects.filter(business=self.business)
    print(f"Existing: {existing}")
    if source_types:
        existing = existing.filter(source_type__in=source_types)
    print(f"Existing: {existing}")
    existing_map = {(obj.source_type, obj.source_id): obj for obj in existing}
    created = 0
    updated = 0
    skipped = 0

    for chunk in all_chunks:
        if len(chunk["content"]) > self.MAX_CHUNK_LENGTH:
            chunk["content"] = chunk["content"][: self.MAX_CHUNK_LENGTH]
        key = (chunk["source_type"], chunk["source_id"])
        existing_chunk = existing_map.get(key)
        content_hash = self._content_hash(chunk["content"])

        metadata = dict(chunk.get("metadata") or {})
        metadata["content_hash"] = content_hash
        metadata["reindex_reason"] = reason

        if existing_chunk and existing_chunk.metadata.get("content_hash") == content_hash:
            skipped += 1
            continue

        embedding = self._embed_text(chunk["content"])
```

Technology Stack

Deliverable 5 — DVC-Orchestrated ML Pipeline

- We implemented a reproducible ML pipeline for post-call analysis using DVC, for software engineering + data science integration.
- We train 3 TF-IDF + Logistic Regression pipelines:
  - category classifier
  - sentiment classifier
  - outcome classifier
- This gives a practical, explainable baseline with fast inference and low operational cost.

dvc.yaml > {} stages > {} train_post_call_models > [ ] outs

dvc.yaml - dvc.yaml file (schema.json) | You, 5 hours ago | 1 author (You)

```yaml
# Post-call ML pipeline. First-time setup: pip install dvc && dvc init
# Regenerate models: dvc repro
stages:
  train_post_call_models:
    cmd: python ml/scripts/train_intent_model.py
    deps:
      - ml/scripts/train_intent_model.py
      - data/call_intent/training.csv
    params:
      - params.yaml:
          - train_intent
    outs:
      - ml/artifacts/post_call_models.joblib
      - ml/artifacts/metrics.json
```

THANK YOU